

Lecture Notes: 1D and 3D Convolutions

CIC1205 - Machine Learning

Eduardo Bezerra

May 28, 2026

0 Contents

Before You Read	2
Learning Objectives	2
1 Generalizing the Convolution Operation	3
1.1 A Unified View of Convolutions	3
2 1D Convolution	4
2.1 Motivation: Sequential and Temporal Data	4
2.2 A Worked Example: 1D Convolution	5
2.3 Multi-Channel 1D Convolution and Parameter Counting	5
2.4 Inductive Biases for Sequential Data	5
2.5 A Concrete Architecture: 1D CNN for Time-Series Classification	6
3 Implementation of 1D CNNs in PyTorch	7
3.1 Using <code>nn.Conv1d</code>	7
3.2 A Comparison: <code>Conv1d</code> , <code>Conv2d</code> , and <code>Conv3d</code> in PyTorch	8
4 3D Convolution	8
4.1 Motivation: Volumetric and Spatiotemporal Data	9
4.2 The 3D Convolution Operation	9
4.3 Parameter Counting for 3D Convolutions	10
4.4 A Worked Example: 3D Convolution on Video	10
4.5 Decomposed 3D Convolutions	11
4.6 Implementation in PyTorch	12
5 Choosing Between 1D, 2D, and 3D Convolutions	14
5.1 A Decision Framework	14
5.2 Mixing Convolution Types	15
6 Common Pitfalls with 1D and 3D Convolutions	15
6.1 Debugging Checklist	15
7 Notable Architectures	16
7.1 1D CNN Architectures	16
7.2 3D CNN Architectures	17
8 Final Remarks	18
8.1 Summary of Key Concepts	18
8.2 Recommended Next Steps	18

0 Before You Read

These notes build directly upon the CNN lecture notes. The following concepts are assumed to be already understood. Review them if necessary before proceeding.

- The 2D convolution operation: kernels, feature maps, padding, stride, and the output-size formula.
- Multi-channel convolution: input shape (C_{in}, H, W) , kernel shape $(C_{\text{out}}, C_{\text{in}}, k, k)$, and parameter counting.
- Pooling layers (max and average) and their effect on spatial dimensions.
- The standard CNN block structure: $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Pool}$.
- The role of the classification head, batch normalization, and data augmentation.
- Basic PyTorch syntax for defining models with `nn.Module` and `nn.Sequential`.

0 Learning Objectives

After studying these notes, you should be able to:

- explain what 1D convolution is, identify the types of data for which it is appropriate, and describe how it differs from 2D convolution geometrically;
- compute the output of a 1D convolutional layer given an input sequence, a kernel, a stride, and a padding value, using the output-size formula;
- describe the three inductive biases that 1D convolutions impose on sequential data (local temporal context, parameter sharing over time, translation equivariance along the time axis);
- implement a 1D CNN for sequence classification in PyTorch using `nn.Conv1d`;
- explain what 3D convolution is, identify the types of data for which it is appropriate, and describe how the third dimension is used;
- apply the output-size formula to all three dimensions of a 3D convolutional layer and count its parameters;
- discuss the computational cost of 3D convolutions and describe common strategies that reduce it (pseudo-3D, (2+1)D decomposition, 3D depthwise separable convolution);
- implement a basic 3D CNN for video classification in PyTorch using `nn.Conv3d`;
- choose the appropriate convolution type (1D, 2D, or 3D) for a given input modality and justify the choice.

1 Generalizing the Convolution Operation

1.1 A Unified View of Convolutions

The 2D convolution studied in the previous notes is a special case of a more general pattern: *slide a small kernel over a structured input, compute a dot product at each position, and produce a feature map with the same structural topology as the input*. The dimensionality of the convolution (1D, 2D, or 3D) refers to the number of spatial axes along which the kernel slides.

- **1D convolution:** the kernel slides along a single axis. Input is a sequence (e.g., audio waveform, time-series sensor data, text after embedding). Output is a shorter or equal-length sequence of features.
- **2D convolution:** the kernel slides along two axes (height and width). Input is an image or a spatial feature map. This is the case covered in the previous notes.
- **3D convolution:** the kernel slides along three axes simultaneously. The third axis is typically time (video frames) or depth (volumetric data such as CT scans).

The same three inductive biases apply in every case: *local connectivity* along the relevant axes, *parameter sharing* across all positions, and *translation equivariance* along those axes. What changes is only the geometry of the input and the direction in which the kernel moves.

Table 1 summarizes the three variants side by side.

Table 1: Comparison of 1D, 2D, and 3D convolutions: input shape, kernel shape, output shape, and typical use cases.

Type	Input shape	Kernel shape	Output shape	Typical use
1D	(C_{in}, L)	$(C_{\text{out}}, C_{\text{in}}, k)$	(C_{out}, L')	Audio, time series, NLP
2D	(C_{in}, H, W)	$(C_{\text{out}}, C_{\text{in}}, k, k)$	(C_{out}, H', W')	Images
3D	(C_{in}, D, H, W)	$(C_{\text{out}}, C_{\text{in}}, k_d, k_h, k_w)$	$(C_{\text{out}}, D', H', W')$	Video, volumetric

The Output-Size Formula Is Universal

The output-size formula derived for 2D convolutions applies independently to each axis:

$$L_{\text{out}} = \left\lfloor \frac{L + 2p - k}{s} \right\rfloor + 1.$$

For 1D convolutions there is one such computation (along the length axis L). For 2D, there are two (height and width). For 3D, there are three (depth, height, and width). Each axis uses its own values of p , k , and s , which may differ across axes.

Entry Ticket 1

Before reading further, consolidate what you know from 2D convolutions.

1. A 2D convolutional layer has input shape $(3, 32, 32)$, uses 64 filters of size 3×3 , padding 1, stride 1. Write down the output shape and the number of learnable parameters.
2. Suppose you have a temperature sensor recording one value per minute for one hour. The data has shape $(1, 60)$ (one channel, 60 time steps). Intuitively, why might a convolutional filter of size $k = 5$ be useful for this data? What would it capture?
3. What would it mean, intuitively, for a 3D convolutional filter to slide over a sequence of video frames? What is it looking for?

2 1D Convolution

2.1 Motivation: Sequential and Temporal Data

Many practical datasets are not spatial grids but *sequences*: the data is indexed along a single axis that represents time, position in a sentence, or position along a physical signal. Examples include:

- *Audio waveforms*: a mono audio clip sampled at 16 kHz is a sequence of 16,000 amplitude values per second.
- *Time-series signals*: ECG recordings, accelerometer readings, stock prices, weather observations.
- *Text after tokenization*: a sentence represented as a sequence of word embeddings, each of dimension d , forming a matrix of shape (d, L) where L is the sentence length.
- *DNA sequences*: a nucleotide sequence encoded as a one-hot vector per position.

An MLP applied to such data faces the same problems as an MLP applied to images: it ignores the ordering and locality of the sequence, and requires far too many parameters for long inputs. A 1D CNN addresses both issues by sliding a kernel along the sequence, capturing *local temporal patterns* at every position.

1D Convolution

Given a multi-channel input sequence $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times L}$ and a kernel $\mathbf{K} \in \mathbb{R}^{C_{\text{in}} \times k}$, the output sequence $\mathbf{Z} \in \mathbb{R}^{L'}$ for a single output channel is:

$$Z_j = \sum_{c=0}^{C_{\text{in}}-1} \sum_{m=0}^{k-1} X_{c, j \cdot s + m} \cdot K_{c, m} + b,$$

where s is the stride, b is a bias term, and j indexes the output position. With C_{out} kernels, the full output has shape (C_{out}, L') , where $L' = \lfloor (L + 2p - k)/s \rfloor + 1$.

2.2 A Worked Example: 1D Convolution

Consider a single-channel input sequence of length 6 and a kernel of size 3 (no padding, stride 1):

$$\mathbf{x} = [1, 3, 2, 4, 0, 5], \quad \mathbf{k} = [1, -1, 2].$$

The output length is $(6 - 3 + 1)/1 = 4$. Computing each output value:

$$\begin{aligned} Z_0 &= (1)(1) + (3)(-1) + (2)(2) = 1 - 3 + 4 = 2, \\ Z_1 &= (3)(1) + (2)(-1) + (4)(2) = 3 - 2 + 8 = 9, \\ Z_2 &= (2)(1) + (4)(-1) + (0)(2) = 2 - 4 + 0 = -2, \\ Z_3 &= (4)(1) + (0)(-1) + (5)(2) = 4 - 0 + 10 = 14. \end{aligned}$$

The output sequence is $\mathbf{z} = [2, 9, -2, 14]$.

Figure 1 illustrates the sliding window mechanism for 1D convolution.

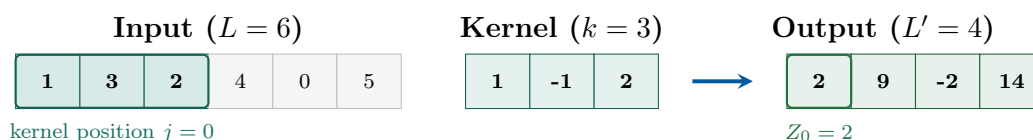


Figure 1: 1D convolution with a kernel of size $k = 3$ applied to a sequence of length $L = 6$ (no padding, stride 1). The kernel is positioned at $j = 0$ (highlighted), computing $Z_0 = (1)(1) + (3)(-1) + (2)(2) = 2$. It then slides one step to the right, and so on, producing the four-element output sequence.

2.3 Multi-Channel 1D Convolution and Parameter Counting

Exactly as in the 2D case, the input to a 1D convolutional layer is typically multi-channel. For example, a 6-channel EEG recording with L time steps has shape $(6, L)$. Each output filter spans all input channels simultaneously.

A 1D convolutional layer with C_{in} input channels, C_{out} output channels (filters), and kernel size k has:

$$C_{\text{out}} \times (C_{\text{in}} \times k + 1) \quad \text{learnable parameters.}$$

For example, a layer with $C_{\text{in}} = 6$, $C_{\text{out}} = 32$, and $k = 5$ has $32 \times (6 \times 5 + 1) = 32 \times 31 = 992$ parameters. This is dramatically fewer than a fully connected layer mapping the same input to 32 features over any reasonably long sequence.

2.4 Inductive Biases for Sequential Data

The three structural properties that 1D CNNs impose on sequential data mirror those of 2D CNNs for images, re-interpreted in the temporal domain:

- **Local temporal context.** Each output feature depends only on a window of k consecutive time steps. This is appropriate for many real-world signals where patterns (a spoken phoneme, an ECG heartbeat, an n-gram) span a limited number of steps.
- **Parameter sharing over time.** The same kernel detects the same local pattern regardless of when it appears in the sequence. A filter that detects a rising slope will fire whether the slope occurs at second 2 or second 20.
- **Translation equivariance along time.** Shifting the input signal by a few steps shifts the feature map by the same number of steps, but does not change the type of features detected.

1D CNNs vs. Recurrent Networks for Sequences

Both 1D CNNs and Recurrent Neural Networks (RNNs/LSTMs) are designed for sequential data, but they process it differently. A 1D CNN processes all positions in parallel using fixed-size local windows, while an RNN processes positions one at a time, maintaining a hidden state that can theoretically carry information from any earlier step. 1D CNNs are generally faster to train (no sequential dependency) and work well when the relevant patterns are short and local. RNNs and Transformers can capture long-range dependencies more explicitly, at higher computational cost. In practice, many tasks (e.g., keyword spotting, ECG classification) are solved effectively and efficiently with 1D CNNs.

2.5 A Concrete Architecture: 1D CNN for Time-Series Classification

Figure 2 shows the general structure of a 1D CNN for classifying a multi-channel time-series signal. The structure directly mirrors the 2D CNN: convolutional blocks reduce the length of the sequence while increasing the number of feature channels, followed by a global aggregation step and a fully connected classification head.

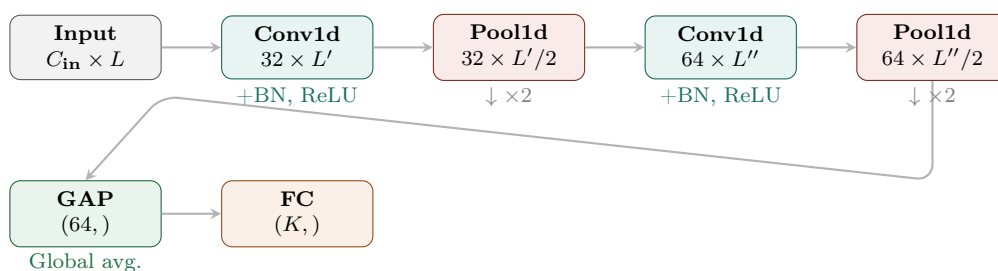


Figure 2: General structure of a 1D CNN for time-series classification. Two convolutional blocks reduce the sequence length while increasing the channel depth. Global average pooling (GAP) aggregates each feature map into a single value, and a fully connected layer maps the resulting vector to class scores. K is the number of classes.

Entry Ticket 2

Work through the following before reading about 3D convolutions.

1. An ECG recording has shape (12, 5000) (12 leads, 5000 samples at 500 Hz). A 1D convolutional layer applies 64 filters of size $k = 7$, with padding 3 and stride 1. What is the output shape? How many learnable parameters does the layer have?
2. Why is global average pooling commonly used at the end of a 1D CNN instead of flattening? What advantage does it offer when the input sequence length L varies between examples?
3. A 1D CNN and an MLP are both applied to classify ECG signals of length 5000. The MLP has one hidden layer of 128 neurons. How many parameters does the MLP's first weight matrix have? Compare this to the 1D CNN layer from question 1. What does this tell you about sample efficiency?

3 Implementation of 1D CNNs in PyTorch

3.1 Using `nn.Conv1d`

In PyTorch, 1D convolution is implemented by `nn.Conv1d`, which mirrors the `nn.Conv2d` API exactly, except that spatial dimensions are collapsed to a single length dimension. The input tensor has shape (N, C_{in}, L) , where N is the batch size.

Listing 1 implements a 1D CNN for classifying multi-channel time-series inputs (e.g., 12-lead ECG) into K classes.

```

1 import torch
2 import torch.nn as nn
3
4 class CNN1D(nn.Module):
5     def __init__(self, in_channels=12, num_classes=5):
6         super().__init__()
7
8         self.features = nn.Sequential(
9             # Block 1: (12, L) -> (32, L) -> (32, L/2)
10             nn.Conv1d(in_channels=in_channels, out_channels=32,
11                     kernel_size=7, padding=3),
12             nn.BatchNorm1d(32),
13             nn.ReLU(),
14             nn.MaxPool1d(kernel_size=2, stride=2),
15
16             # Block 2: (32, L/2) -> (64, L/2) -> (64, L/4)
17             nn.Conv1d(in_channels=32, out_channels=64,
18                     kernel_size=5, padding=2),
19             nn.BatchNorm1d(64),
20             nn.ReLU(),
21             nn.MaxPool1d(kernel_size=2, stride=2),
22
23             # Block 3: (64, L/4) -> (128, L/4) -> (128, L/8)
24             nn.Conv1d(in_channels=64, out_channels=128,

```



```

25         kernel_size=3, padding=1),
26         nn.BatchNorm1d(128),
27         nn.ReLU(),
28         nn.MaxPool1d(kernel_size=2, stride=2),
29     )
30
31     # Global average pooling: (128, L/8) -> (128,)
32     self.gap = nn.AdaptiveAvgPool1d(output_size=1)
33
34     # Classification head
35     self.classifier = nn.Sequential(
36         nn.Flatten(),          # (128, 1) -> (128,)
37         nn.Linear(128, 64),
38         nn.ReLU(),
39         nn.Dropout(p=0.3),
40         nn.Linear(64, num_classes),
41     )
42
43     def forward(self, x):
44         x = self.features(x)
45         x = self.gap(x)
46         x = self.classifier(x)
47         return x
48
49 model = CNN1D(in_channels=12, num_classes=5)
50 # Input: batch of 16 ECG recordings, 12 channels, 5000 samples
51 x = torch.randn(16, 12, 5000)
52 print(model(x).shape)    # -> torch.Size([16, 5])

```

Listing 1: 1D CNN for multi-channel time-series classification in PyTorch.

AdaptiveAvgPool1d for Variable-Length Inputs

`nn.AdaptiveAvgPool1d(output_size=1)` computes the mean over the entire length dimension, regardless of the input length. This is the 1D equivalent of global average pooling. By using it, the model accepts sequences of any length at inference time without changing the architecture. Without it, a `Flatten` layer would produce vectors of different sizes for different input lengths, making the fully connected layer incompatible.

3.2 A Comparison: Conv1d, Conv2d, and Conv3d in PyTorch

Table 2 summarizes the API differences between the three convolution types in PyTorch. The parameters are identical in name; only the dimensionality of the tensors and the tuples used for kernel size, padding, and stride differ.

4 3D Convolution

Table 2: API comparison of `nn.Conv1d`, `nn.Conv2d`, and `nn.Conv3d` in PyTorch.

Class	Input shape	kernel_size	Example call
<code>nn.Conv1d</code>	(N, C_{in}, L)	integer or $(k,)$	<code>Conv1d(8, 16, kernel_size=5)</code>
<code>nn.Conv2d</code>	(N, C_{in}, H, W)	integer or (kH, kW)	<code>Conv2d(3, 64, kernel_size=3)</code>
<code>nn.Conv3d</code>	$(N, C_{\text{in}}, D, H, W)$	integer or (kD, kH, kW)	<code>Conv3d(3, 32, kernel_size=(3,3,3))</code>

4.1 Motivation: Volumetric and Spatiotemporal Data

Some data modalities are inherently three-dimensional in the sense that meaningful patterns extend across three axes simultaneously:

- **Video:** a video clip is a sequence of image frames. A single frame is 2D (height $\times$ width); the full clip adds a temporal axis, giving a tensor of shape (C, T, H, W) , where T is the number of frames.
- **Volumetric medical imaging:** CT (computed tomography) and MRI scans are stacks of 2D slices, forming a 3D volume (C, D, H, W) , where D is the depth (number of slices). Patterns such as tumors must be detected across all three spatial dimensions simultaneously.
- **3D point clouds and voxel grids:** 3D objects represented on a regular voxel grid, used in robotics and autonomous driving.
- **Hyperspectral imaging:** remote sensing data with many spectral bands that can be treated as a depth axis.

4.2 The 3D Convolution Operation

A 3D convolutional layer extends the 2D operation by adding a third dimension (depth or time) along which the kernel also slides.

3D Convolution

Given a single-channel 3D input $\mathbf{X} \in \mathbb{R}^{D \times H \times W}$ and a kernel $\mathbf{K} \in \mathbb{R}^{k_d \times k_h \times k_w}$, the output $\mathbf{Z} \in \mathbb{R}^{D' \times H' \times W'}$ for one output channel is:

$$Z_{i,j,l} = \sum_{p=0}^{k_d-1} \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X_{i+p, j+m, l+n} \cdot K_{p,m,n} + b.$$

With C_{in} input channels, the kernel has shape $(C_{\text{in}}, k_d, k_h, k_w)$ and the summation additionally runs over all channels. With C_{out} such kernels, the output has shape $(C_{\text{out}}, D', H', W')$.

The output size along each axis is computed independently using the same formula:

$$D' = \left\lfloor \frac{D + 2p_d - k_d}{s_d} \right\rfloor + 1, \quad H' = \left\lfloor \frac{H + 2p_h - k_h}{s_h} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2p_w - k_w}{s_w} \right\rfloor + 1.$$

4.3 Parameter Counting for 3D Convolutions

A 3D convolutional layer with C_{in} input channels, C_{out} output channels, and kernel size $k_d \times k_h \times k_w$ has:

$$C_{\text{out}} \times (C_{\text{in}} \times k_d \times k_h \times k_w + 1) \quad \text{learnable parameters.}$$

The Computational Cost of 3D Convolutions

Compared to a 2D convolutional layer with the same spatial kernel size $k \times k$, a 3D layer with an additional temporal kernel size k_d has k_d times as many parameters *per filter* and k_d times as many multiply-accumulate operations per output position. For video, typical values are $T = 8$ –32 frames and $k_d = 3$, so the computational overhead relative to processing a single frame is substantial. This motivates the decomposed architectures discussed in Section 4.5.

4.4 A Worked Example: 3D Convolution on Video

Consider a short video clip with $C_{\text{in}} = 3$ (RGB) channels, $T = 8$ frames, and spatial size $H = W = 112$. The input has shape $(3, 8, 112, 112)$. A 3D convolutional layer applies $C_{\text{out}} = 64$ filters of size $k_d \times k_h \times k_w = 3 \times 3 \times 3$ with padding $(1, 1, 1)$ and stride $(1, 1, 1)$.

Output shape:

$$D' = \frac{8 + 2 - 3}{1} + 1 = 8, \quad H' = \frac{112 + 2 - 3}{1} + 1 = 112, \quad W' = \frac{112 + 2 - 3}{1} + 1 = 112.$$

Output shape: $(64, 8, 112, 112)$.

Parameter count:

$$64 \times (3 \times 3 \times 3 \times 3 + 1) = 64 \times 82 = 5,248.$$

While the parameter count is still small due to weight sharing, the computational cost is proportional to the total number of output values, which is $64 \times 8 \times 112 \times 112 = 64,225,280$, highlighting why computational efficiency is a primary concern in 3D CNN design.

Entry Ticket 3

Work through the following before reading about decomposed 3D convolutions.

1. A CT scan has shape $(1, 64, 256, 256)$ (1 channel, 64 slices, 256×256 pixels per slice). A 3D convolutional layer applies 32 filters of size $3 \times 3 \times 3$ with padding $(1, 1, 1)$ and stride $(1, 1, 1)$. What is the output shape? How many parameters does this layer have?
2. Compared to a 2D convolutional layer applied to a single CT slice with the

- same C_{in} , C_{out} , and spatial kernel size, how many more parameters does the 3D layer have? What additional information does it capture?
3. Why would it be insufficient to apply a 2D CNN independently to each video frame and then aggregate the frame-level predictions for action recognition tasks like “jumping” or “throwing”? What information would be lost?

4.5 Decomposed 3D Convolutions

The high computational cost of full 3D convolution has motivated several factored designs that approximate the full 3D operation at lower cost.

Pseudo-3D (P3D) Convolution

A $(k_d \times k_h \times k_w)$ convolution is approximated by a sequence of two operations:

1. A *spatial convolution*: $(1 \times k_h \times k_w)$, which processes each frame independently, capturing spatial patterns.
2. A *temporal convolution*: $(k_d \times 1 \times 1)$, which operates only along the time axis, capturing temporal patterns.

Parameter comparison (with $C_{\text{in}} = C_{\text{out}} = C$, $k_d = k_h = k_w = 3$):

- Full 3D: $C \times (C \times 3 \times 3 \times 3) = 27C^2$ weights.
- P3D: $C \times (C \times 1 \times 3 \times 3) + C \times (C \times 3 \times 1 \times 1) = 9C^2 + 3C^2 = 12C^2$ weights.

P3D uses approximately 44% of the parameters of the full 3D convolution while modeling both spatial and temporal structure.

(2+1)D Convolution

The (2+1)D convolution [?] explicitly splits the 3D operation into a full 2D spatial convolution followed by a 1D temporal convolution, with a nonlinearity in between:

1. $C'_{\text{out}} \times (C_{\text{in}} \times 1 \times k_h \times k_w + 1)$ parameters for the spatial step (using an intermediate number of channels C'_{out}).
2. $C_{\text{out}} \times (C'_{\text{out}} \times k_d \times 1 \times 1 + 1)$ parameters for the temporal step.

Why Decompose 3D Convolutions?

Decomposed 3D convolutions (P3D, (2+1)D) reduce parameter counts by approximately 50–60% compared to full 3D convolutions with the same kernel size, while maintaining or improving accuracy. The (2+1)D design has the additional benefit of inserting a non-linearity between the spatial and temporal stages, increasing representational capacity relative to a single fused operation. The architecture used in R(2+1)D [?] achieves state-of-the-art results on action recognition benchmarks with substantially lower computational cost than C3D (full 3D convolutions).

3D Depthwise Separable Convolution

Extending the 2D depthwise separable idea (used in MobileNet) to 3D: the operation is split into a *depthwise* convolution (one filter per channel, applied independently) followed by a *pointwise* ($1 \times 1 \times 1$) convolution that mixes channels. For a $3 \times 3 \times 3$ layer:

- Full 3D: $C_{\text{out}} \times C_{\text{in}} \times 27$ weights.
- 3D depthwise separable: $C_{\text{in}} \times 27 + C_{\text{in}} \times C_{\text{out}}$ weights.

Figure 3 compares the three decompositions schematically.

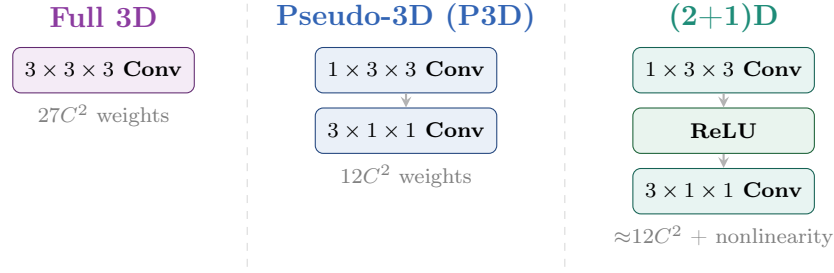


Figure 3: Comparison of full 3D convolution with two decomposed variants: Pseudo-3D (P3D) and (2+1)D convolution. Both decompositions separate spatial ($1 \times 3 \times 3$) and temporal ($3 \times 1 \times 1$) processing. The (2+1)D design inserts a nonlinearity between stages, increasing representational capacity. Both require roughly 44% of the parameters of the full 3D operation for $C_{\text{in}} = C_{\text{out}} = C$ and $k = 3$.

Common Misconception

A 3D convolutional layer applied to video does not process three independent spatial dimensions. The three axes are *temporal*, *height*, and *width*. This means the kernel captures patterns that unfold over time (motion, deformation) as well as patterns that are spatially extended in each frame. Simply stacking 2D convolutions applied frame-by-frame ignores motion patterns, while a 3D (or (2+1)D) kernel can detect them because it spans multiple frames simultaneously.

4.6 Implementation in PyTorch

Listing 2 implements a simple 3D CNN for video clip classification, using `nn.Conv3d`.

```

1 import torch
2 import torch.nn as nn
3
4 class CNN3D(nn.Module):
5     def __init__(self, num_classes=10):
6         super().__init__()
7
8         self.features = nn.Sequential(
9             # Block 1: (3,T,H,W) -> (32,T,H/2,W/2)
10             nn.Conv3d(in_channels=3, out_channels=32,
11                      kernel_size=(3, 3, 3), padding=(1, 1, 1)),
12             nn.BatchNorm3d(32),

```

```

13     nn.ReLU(),
14     # Spatial downsampling; preserve temporal dimension
15     nn.MaxPool3d(kernel_size=(1, 2, 2), stride=(1, 2, 2)),
16
17     # Block 2: (32,T,H/2,W/2) -> (64,T/2,H/4,W/4)
18     nn.Conv3d(in_channels=32, out_channels=64,
19               kernel_size=(3, 3, 3), padding=(1, 1, 1)),
20     nn.BatchNorm3d(64),
21     nn.ReLU(),
22     # Downsample in all three dimensions
23     nn.MaxPool3d(kernel_size=(2, 2, 2), stride=(2, 2, 2)),
24
25     # Block 3: (64,T/2,H/4,W/4) -> (128,T/2,H/8,W/8)
26     nn.Conv3d(in_channels=64, out_channels=128,
27               kernel_size=(3, 3, 3), padding=(1, 1, 1)),
28     nn.BatchNorm3d(128),
29     nn.ReLU(),
30     nn.MaxPool3d(kernel_size=(2, 2, 2), stride=(2, 2, 2)),
31 )
32
33 # Global average pooling over all spatial and temporal dims
34 self.gap = nn.AdaptiveAvgPool3d(output_size=(1, 1, 1))
35
36 self.classifier = nn.Sequential(
37     nn.Flatten(),
38     nn.Linear(128, 256),
39     nn.ReLU(),
40     nn.Dropout(p=0.4),
41     nn.Linear(256, num_classes),
42 )
43
44 def forward(self, x):
45     x = self.features(x)
46     x = self.gap(x)
47     x = self.classifier(x)
48     return x
49
50 model = CNN3D(num_classes=10)
51 # Input: batch of 4 clips, 3 channels, 16 frames, 112x112
52 x = torch.randn(4, 3, 16, 112, 112)
53 print(model(x).shape)    # -> torch.Size([4, 10])

```

Listing 2: 3D CNN for video clip classification in PyTorch. Input: $(N, 3, T, H, W)$ where T is the number of frames.

Asymmetric Pooling for Video

In early video CNN layers, it is common to use pooling kernels that downsample the spatial dimensions but not the temporal dimension (e.g., `MaxPool3d(kernel_size=(1,2,2))`). This preserves temporal resolution while re-

ducing the spatial cost, which is beneficial when the temporal dimension is already short (e.g., 8 or 16 frames). Aggressive temporal pooling in early layers can destroy motion information before it has been encoded into features.

Entry Ticket 4

Consider the 3D CNN defined in Listing 2.

1. Trace the tensor shape through the entire network starting from an input of shape $(4, 3, 16, 112, 112)$. After each block, write down the shape. Check that `AdaptiveAvgPool3d(1,1,1)` produces the expected output.
2. How many learnable parameters does the first `Conv3d` layer have? Compare to what a `Conv2d` layer with the same spatial kernel size and channel sizes would have. What is the ratio?
3. Suppose you want to replace the first `Conv3d` layer with a $(2+1)$ D decomposition: a $1 \times 3 \times 3$ spatial convolution followed by a $3 \times 1 \times 1$ temporal convolution, using an intermediate channel size of 32. Write the two `nn.Conv3d` calls and count the total number of parameters. Compare to the original.

5 Choosing Between 1D, 2D, and 3D Convolutions

5.1 A Decision Framework

The choice of convolution dimensionality should be driven by the structure of the input data and the nature of the patterns to be detected. Table 3 provides a practical guide.

Table 3: Guide for choosing between 1D, 2D, and 3D convolutions based on data type and pattern structure.

Data type	Input shape	Pattern structure	Conv type
Univariate time series	$(1, L)$	Local temporal patterns	1D
Multi-channel signal	(C, L)	Local cross-channel + temporal	1D
Grayscale or RGB image	(C, H, W)	Spatial (edges, textures)	2D
Sequence of image frames (video)	(C, T, H, W)	Spatial + temporal (motion)	3D or $(2+1)$ D
CT / MRI volumetric scan	(C, D, H, W)	Fully spatial (3D anatomy)	3D
Text / NLP embeddings	(d, L)	Local n-gram patterns	1D
Hyperspectral remote sensing	(B, H, W) or (B, D, H, W)	Spectral + spatial	2D or 3D

5.2 Mixing Convolution Types

In practice, architectures often combine different convolution types. For example:

- **SlowFast networks** [?] use two pathways: one processes video at low frame rate with 2D convolutions (capturing appearance), and another processes at high frame rate with 3D or (2+1)D convolutions (capturing motion).
- **Medical image analysis** pipelines often apply 2D CNNs to individual MRI slices for fast feature extraction, then aggregate slice-level features with a 1D CNN or LSTM across the depth dimension.
- **Audio classification** models sometimes apply 2D convolutions to mel-spectrograms (treating the frequency axis as height and the time axis as width) instead of 1D convolutions on the raw waveform.

2D Convolution on Spectrograms vs. 1D on Raw Audio

Both approaches are valid for audio tasks, but they encode different assumptions. A 2D CNN applied to a mel-spectrogram captures patterns in the time-frequency domain (e.g., pitch trajectories, harmonic structures). A 1D CNN on the raw waveform (as in WaveNet or SincNet) operates on the amplitude signal directly. The right choice depends on whether the relevant patterns are best characterized in the time domain or in the frequency domain, and on computational constraints.

General Principle: Match Convolution Topology to Data Topology

The number of convolution dimensions should match the number of structured axes in the input over which local patterns are meaningful and position-invariant. If patterns are local along one axis, use 1D. If local in two, use 2D. If local in three, use 3D. When the three-axis cost is prohibitive, decomposed operations (P3D, (2+1)D) offer a practical compromise.

6 Common Pitfalls with 1D and 3D Convolutions

6.1 Debugging Checklist

Many of the pitfalls from 2D CNNs apply directly. Table 4 lists issues specific to 1D and 3D convolutions.

Inflating 2D Weights for Video Initialization

A common technique for initializing 3D video models is *weight inflation* [?]: start from a 2D CNN pre-trained on ImageNet, and expand each $k_h \times k_w$ kernel to $k_d \times k_h \times k_w$ by replicating its values along the new temporal axis and dividing by k_d . This initialization encodes the prior that, before seeing any video data, the network should process each frame similarly to how the 2D model processes a still image. Fine-tuning then adapts the temporal filters to video-specific patterns.

Table 4: Common pitfalls specific to 1D and 3D convolutions, with fixes.

Symptom		Likely cause	Fix
<code>RuntimeError</code> on <code>Conv1d</code>		Input shape is (N, L) instead of (N, C, L)	Add channel dim: <code>x.unsqueeze(1)</code>
Wrong output length		Forgot padding; wrong stride	Apply output-size formula before coding
Memory error on 3D CNN		Input resolution or T too large	Reduce spatial/temporal resolution; use (2+1)D
Temporal patterns not learned		Temporal dim pooled too early	Use <code>MaxPool3d(1,2,2)</code> in early layers
Slow 3D training		Full 3D kernels throughout	Replace with (2+1)D or P3D blocks
Linear size mismatch after 3D features		GAP output shape mis-computed	Use <code>AdaptiveAvgPool3d(1,1,1)</code>
Model generalizes to image but not to video		2D pretrained weights applied naively	Inflate 2D weights along temporal axis (see note below)

Common Misconception

Using `nn.Conv3d` on video data does not automatically mean the model will learn useful temporal features. If all pooling layers aggressively downsample the temporal axis from the start, or if the training dataset is too small, the temporal filters may remain near zero and the model effectively behaves as a 2D CNN applied to a single frame. Always verify that temporal pooling is applied conservatively in early layers, and that the training data contains diverse motion patterns.

7 Notable Architectures

7.1 1D CNN Architectures

- **WaveNet** [?]: a 1D dilated causal CNN for raw audio generation, using *dilated convolutions* (also called *atrous convolutions*) to achieve exponentially growing receptive fields without increasing the number of parameters.
- **TCN (Temporal Convolutional Network)** [?]: a 1D CNN with dilated causal convolutions and residual connections, demonstrated to match or outperform RNNs on many sequence modeling benchmarks.
- **SincNet** [?]: a 1D CNN for speaker recognition that constrains the first layer's filters to be sinc functions, injecting known signal-processing knowledge into the

architecture.

7.2 3D CNN Architectures

- **C3D** [?]: one of the first successful deep 3D CNNs for video understanding, using $3 \times 3 \times 3$ convolutions throughout. Established that spatiotemporal features from 3D CNNs transfer across video tasks.
- **I3D** [?]: a two-stream 3D CNN (optical flow + RGB) initialized by inflating a 2D Inception network. Achieved state-of-the-art results on action recognition benchmarks.
- **R(2+1)D** [?]: a ResNet-based architecture replacing 3D convolutions with (2+1)D decompositions. Demonstrated that the decomposition improves both efficiency and accuracy.
- **SlowFast** [?]: a two-pathway architecture combining a slow 2D stream (appearance) with a fast 3D stream (motion), achieving high accuracy with fewer parameters than single-stream 3D models.
- **U-Net (3D)** [?]: the 3D extension of the U-Net segmentation architecture, widely used for volumetric medical image segmentation.

Dilated (Atrous) Convolutions

Dilated convolutions introduce a *dilation rate* r : the kernel is applied with gaps of $r - 1$ elements between each pair of kernel weights. A kernel of size k with dilation rate r has an effective receptive field of $k + (k - 1)(r - 1)$ without increasing the number of parameters. For 1D CNNs, doubling the dilation rate in successive layers (e.g., $r = 1, 2, 4, 8, \dots$) yields an exponentially growing receptive field. Dilated convolutions are supported in PyTorch via the `dilation` argument to `nn.Conv1d`, `nn.Conv2d`, and `nn.Conv3d`.

Entry Ticket 5

Reflect on when to use each convolution type.

1. You are building a system to classify spoken commands (“yes”, “no”, “stop”, etc.) from 1-second mono audio clips sampled at 16 kHz. Would you prefer to apply (a) a 1D CNN on the raw waveform, (b) a 2D CNN on the mel-spectrogram, or (c) a 3D CNN? Justify your answer and describe the input shape for each option.
2. A hospital wants to detect pulmonary nodules in CT scans. Each scan has shape $(1, 128, 512, 512)$. Describe a strategy to use 3D convolutions efficiently, taking into account the memory constraints of a GPU with 24 GB of RAM.
3. A colleague suggests applying a 1D CNN to video by reshaping each frame into a 1D vector and treating the video as a sequence of T such vectors. What structural assumption does this break compared to using a 3D CNN? Under what conditions might this approximation still work?

8 Final Remarks

8.1 Summary of Key Concepts

The following points summarize the essential content of these notes:

- The convolution operation generalizes to any number of dimensions. The choice of dimensionality should match the number of structured axes in the input over which local patterns are meaningful.
- **1D convolution** slides a kernel along a single axis (time, position in a sequence). It is appropriate for audio waveforms, time-series data, and text embeddings. The output-size formula, parameter counting, and three inductive biases (local context, parameter sharing, translation equivariance) are identical in principle to the 2D case, reduced by one dimension.
- A 1D convolutional layer with C_{in} input channels, C_{out} output channels, and kernel size k has $C_{\text{out}} \times (C_{\text{in}} \times k + 1)$ parameters.
- **3D convolution** slides a kernel along three axes simultaneously (depth/time, height, width). It is appropriate for video and volumetric data where patterns span multiple frames or slices. The output size along each axis is computed independently using the standard formula.
- A 3D convolutional layer with kernel $k_d \times k_h \times k_w$ has $C_{\text{out}} \times (C_{\text{in}} \times k_d \times k_h \times k_w + 1)$ parameters. Computational cost scales with the product of the three kernel dimensions.
- **Decomposed 3D convolutions** (Pseudo-3D, (2+1)D, depthwise separable 3D) approximate the full 3D operation by separating spatial and temporal processing, reducing parameter counts by roughly 50–60% while preserving accuracy.
- In PyTorch, `nn.Conv1d`, `nn.Conv2d`, and `nn.Conv3d` share the same API; only the dimensionality of input tensors and parameter tuples differs. `AdaptiveAvgPool1d` and `AdaptiveAvgPool3d` with `output_size=1` implement global average pooling for variable-length inputs.
- **Weight inflation** is a common initialization strategy for 3D video models: 2D pre-trained weights are expanded along the temporal axis, providing a strong starting point before fine-tuning on video data.
- **Dilated convolutions** increase the receptive field exponentially without increasing parameters, by introducing gaps between kernel elements. They are available in all three convolution classes via the `dilation` argument.

8.2 Recommended Next Steps

These notes prepare the student for the following topics:

MLPs $\rightarrow$ 2D CNNs $\rightarrow$ **1D & 3D CNNs** $\rightarrow$ Sequence Models (RNN/LSTM) $\rightarrow$ Attention & Transformers $\rightarrow$ Generative Models

Key Takeaways

- The convolution topology should match the data topology: 1D for sequences, 2D for images, 3D for volumes and video.
- 1D CNNs are a practical, efficient alternative to RNNs for tasks where relevant patterns are local in time, and they are widely used in audio, ECG, and sensor data classification.
- 3D CNNs capture spatiotemporal patterns in video and volumetric data, at significantly higher computational cost than 2D CNNs. Decomposed operations (P3D, (2+1)D) offer a practical efficiency–accuracy trade-off.
- The same training pipeline (backpropagation, batch normalization, data augmentation, transfer learning) applies to 1D and 3D CNNs as to 2D CNNs. The differences are entirely in the geometry of the convolution, not in the learning algorithm.
- Understanding the 2D case thoroughly is sufficient to derive the 1D and 3D cases: the output-size formula, parameter counting, and inductive biases all generalize by simply changing the number of dimensions.

8 References
